School App — Release-Readiness Documentation

> **Scope & method.** This document reports the **actual current state** of the code on branch `claude/angry-nash-cf956e`, verified by reading `pubspec.yaml`, `android/app/build.gradle`, `android/settings.gradle`, `AndroidManifest.xml`, `firestore.rules`, `firebase.json`, `lib/main.dart`, `lib/theme.dart`, and by scanning the full `lib/` tree. `SPEC.md` (dated 2026-05-11) and `CLAUDE.md` were read for intent, but the code has since moved well past them — **where the docs and the code disagree, the code is reported as truth** and the divergence is called out.
>
> Generated 2026-05-29. Read-only audit — no code was modified.

1. App Overview

Field	Value
Display name (Android label)	`School App`
Flutter package name	`school_app` (`pubspec.yaml`)
applicationId / package	`com.example.school_app` — ■■ **still the default placeholder** (see §7)
Android namespace	`com.example.school_app`
versionName	`1.0.0` (`pubspec.yaml` `version: 1.0.0+1`, mirrored in `android/local.properties`)
versionCode	`1`
Firebase project	`attendanceapp-e76e1`

Purpose. School App is a multi-role school-management mobile app for a single school (with partial multi-tenant scaffolding). It centralises daily attendance, timetables and substitutions, homework, copy-checking, exams and report cards, fee structure/collection, announcements/notifications, a photo gallery, staff-task and meeting workflows, a principal end-of-day digest, and a parent (guardian) portal. Data lives in Cloud Firestore; auth is Firebase Auth (email/password + phone OTP for guardians).

Intended users. Six roles: `owner`, `principal` (and a combined `ownerPrincipal`), `coordinator`, `teacher` / `subjectTeacher`, and `guardian` (parent). Owners/admins manage the school and accounts; coordinators run timetable/substitution/announcements; teachers handle their classes; guardians view their own child's data.

> **Doc divergence:** `CLAUDE.md` and `SPEC.md` state "**There is no Firebase Auth**… Login is purely Firestore-based." This is **no longer true.** `lib/main.dart`, `lib/services/auth_service.dart`, and `lib/screens/login_screen.dart` all use `FirebaseAuth` (email/password sign-in, password reset, phone OTP for guardians). `allowed_users` is now keyed by Firebase Auth UID and holds role/schoolId metadata.

2. Tech Stack

- **Flutter:** 3.27.4 (engine `82bd5b7209`); channel reported as `user-branch`.
- **Dart:** 3.6.2. `pubspec.yaml` SDK constraint: `>=3.3.4 <4.0.0`.
- **Android Gradle Plugin:** 8.3.2 · **Kotlin:** 1.9.23 · **google-services:** 4.4.1 (`android/settings.gradle`).

Runtime dependencies (exact, from `pubspec.yaml`)

Package	Version	Notes
`shared_preferences`	`^2.2.2`	Session, offline attendance queue, digest cache
`provider`	`^6.1.1`	State management (`SchoolSettingsProvider`)
`table_calendar`	`^3.1.2`	Attendance history / calendar views
`path_provider`	`^2.1.2`	Temp dirs for PDF/CSV export

Package	Version	Notes
`http`	`^1.2.1`	Generic HTTP
`google_mlkit_text_recognition`	`^0.12.0`	OCR (text recognition) — heavy native dep; verify it's still used
`google_sign_in`	`^6.2.1`	■■ **No usages found in** `lib/` — appears to be a dead dependency
`image_picker`	`^1.1.2`	Gallery photo capture/upload
`url_launcher`	`^6.3.0`	`tel:` dial in Daily Calls, WhatsApp links
`font_awesome_flutter`	`10.6.0`	Icons (pinned, no caret)
`pdf`	`^3.10.8`	Report cards, certificates, digest PDFs
`printing`	`^5.12.0`	Print/share PDFs
`firebase_core`	`^2.27.0`	Firebase init
`firebase_auth`	`^4.17.0`	**Now actively used** (email/pw + phone OTP)
`cloud_firestore`	`^4.17.0`	All data storage
`firebase_storage`	`^11.7.0`	Gallery photo storage
`fl_chart`	`^0.68.0`	Analytics charts
`connectivity_plus`	`^6.0.3`	Offline detection
`file_picker`	`^8.0.5`	CSV import
`csv`	`^6.0.0`	CSV export/import
`flutter_image_compress`	`^2.1.0`	Gallery compression
`image`	`^4.1.3`	Watermarking
`image_gallery_saver`	`^2.0.3`	Save photos to device
`share_plus`	`^7.2.1`	Share PDFs/reports
`cached_network_image`	`^3.3.1`	Gallery image caching
`shimmer`	`^3.0.0`	Loading skeletons
`crypto`	`^3.0.3`	Hashing (e.g. password/consent hashing)
`cupertino_icons`	`^1.0.6`	iOS-style icons

Dev dependencies: `flutter_lints: ^3.0.0`, `flutter_test`.

Outdated / deprecated flags

- `google_sign_in` **(`^6.2.1`)** — present in `pubspec.yaml` but **not imported anywhere in** `lib/` (`AuthService.signInWithGoogle()` is a stub returning `null`). Dead weight; remove or wire up.
- `image_gallery_saver` **(`^2.0.3`)** — this package is effectively unmaintained on pub.dev; verify it still builds against the target SDK or migrate to `gal`/`saver_gallery`.
- **`firebase_*` 2.x / 4.x / 11.x series** — these are not the latest major lines. Functional, but a pre-launch dependency review/upgrade pass is advisable.
- **No FCM / Analytics / Crashlytics** packages are present (confirmed against `pubspec.lock`). There is dead FCM-token code in `firestore_service.dart` (see §9) but no messaging package backs it.

3. Project Structure

State management: `provider` for one cross-cutting concern (`SchoolSettingsProvider`), but the dominant pattern is **singleton services wrapping Firestore** (`ServiceName()` factories) consumed directly by `StatefulWidget` screens via `FutureBuilder` / `setState`. There is no global store, BLoC, or Riverpod.

```
lib/
■■■ main.dart                # Firebase init + _SplashGate session router
```

```
■■■ theme.dart                # AppTheme — single colour source (Material 2)
■■■ firebase_options.dart     # generated (android only)
■■■ data/
■   ■■■ student_data.dart
■   ■■■ template_seeds.dart
■■■ models/                   # 30 plain-Dart models (toJson/fromJson, no codegen)
■   ■■■ student.dart, teacher.dart, attendance_status.dart, exam.dart, fee.dart …
■   ■■■ staff_task.dart, meeting.dart, todo_item.dart, calendar_event.dart …
■   ■■■ app_user.dart, school.dart, school_onboarding.dart, parental_consent.dart …
■■■ providers/
■   ■■■ school_settings_provider.dart
■■■ repositories/             # student_repository(+_fake) — partial repo layer
■■■ services/                 # 33 singleton services (see §6)
■   ■■■ base_firestore_service.dart  # currentSchoolId + school-scoped helpers
■   ■■■ auth_service.dart, timetable_service.dart, student_service.dart …
■   ■■■ audit_log_service.dart, meeting_service.dart, consent_service.dart …
■■■ screens/                  # ~98 screen files (see §5)
■   ■■■ owner/                # owner_home, owner_principal_home, edit_school_settings
■   ■■■ onboarding/           # 6-step school onboarding wizard
■   ■■■ gallery/              # album grid, detail, fullscreen viewer
■   ■■■ meeting/              # coordinator/principal/teacher meeting screens
■   ■■■ tasks/                # staff-task create/detail/analytics
■   ■■■ admin/                # audit_log_screen
■   ■■■ coordinator/          # absent_teachers, report_card_template_editor
■   ■■■ consent/ , birthdays/
■   ■■■ *.dart                # role dashboards + feature screens
■■■ scripts/                  # migrate_users.dart, migrate_to_multitenant.dart (one-off)
■■■ utils/                    # role_guard, privacy_notice, report_card_pdf_builder
■■■ widgets/                  # consent_pending_banner
```

Note: `lib/repositories/` introduces a half-finished repository abstraction (`student_repository.dart` + a `_fake` variant) that coexists with the dominant direct-service pattern — an inconsistency, not a blocker.

4. Features by Role

Status legend: ■ implemented (screen present, substantial code) · ■ partial / duplicate / unverified at runtime · ■ stub/placeholder.

Status is judged from source presence and size; it is **not** runtime-verified (this was a read-only audit).

Owner (`owner`) / Owner-Principal (`ownerPrincipal`)

Feature	Screen(s)	Status
Owner home / school dashboard	`owner/owner_home.dart` (1925 LoC)	■
Combined owner+principal home	`owner/owner_principal_home.dart` (1176 LoC)	■
Edit school settings / branding	`owner/edit_school_settings_screen.dart` (1055 LoC)	■
Manage features / account creation & deletion	`owner_home.dart` (recent commits stamp `schoolId` on created accounts)	■
Multi-school onboarding wizard (6 steps)	`onboarding/school_onboarding_screen.dart` + `step1…step6`	■ wizard present; multi-tenant write path still falls back to hardcoded `school_1` (§9)

Principal (`principal`)

Feature	Screen(s)	Status
Principal dashboard	`principal_dashboard.dart`	■
End-of-day digest + PDF	`principal_digest_screen.dart` / `principal_digest_service.dart`	■
Staff task management (create/assign)	`staff_task_management_screen.dart`, `tasks/create_staff_task_screen.dart`, `tasks/staff_task_detail_screen.dart`, `tasks/staff_task_analytics_view.dart`	■

Feature	Screen(s)	Status
Meeting records	`meeting/principal_meeting_records_screen.dart`, `meeting/meeting_detail_screen.dart`	■
Analytics (attendance trends, absence leaders)	`analytics_screen.dart`	■
Leave approvals	`leave_requests_screen.dart`	■
Admin utilities	`admin_screen.dart`, `admin/audit_log_screen.dart`	■
Student deletion requests review	`student_deletion_requests_screen.dart`	■
Report cards	`report_card_screen.dart` + `utils/report_card_pdf_builder.dart`	■

Coordinator (`coordinator`)

Feature	Screen(s)	Status
Coordinator dashboard	`coordinator_dashboard.dart`	■
Timetable settings (bells/classes)	`timetable_settings_screen.dart`	■
Substitution planning + history	`substitution_plan_screen.dart`, `substitution_history_screen.dart`, `free_bells_screen.dart`, `coordinator/absent_teachers_screen.dart`	■
Leave requests review	`leave_requests_screen.dart`, `student_leave_requests_screen.dart`	■
Teacher management	`teacher_management_screen.dart`	■
Duties roster	`assign_duties_screen.dart`	■
Announcements	`announcements_screen.dart`	■
Exams / copy-check overview	`exam_management_screen.dart`, `copy_check_overview_screen.dart`	■
Fee structure & collection	`fee_structure_screen.dart`, `fee_collection_screen.dart`, `fee_overview_screen.dart`	■
Report-card template editor	`coordinator/report_card_template_editor.dart`	■
Meeting records	`meeting/coordinator_meeting_records_screen.dart`	■
Coordinator staff tasks	`coordinator_staff_tasks_screen.dart`	■
Add student / class picker	`add_student_screen.dart`, `class_picker_screen.dart`	■
To-do list + reminder banner	`todo_list_screen.dart`, `todo_reminder_banner.dart`	■

Teacher (`teacher` / `subjectTeacher`)

Feature	Screen(s)	Status
Teacher home	`home_screen.dart` (was flagged as a merge conflict in SPEC.md; **conflict appears resolved** — see §9)	■
Mark attendance (+ offline queue)	`attendance_screen.dart` + `offline_queue_service.dart`	■
Attendance history (calendar)	`attendance_history_screen.dart`	■
Homework post/manage	`homework_screen.dart`, `homework_overview_screen.dart`	■

Feature	Screen(s)	Status
Copy checking	`copy_checking_screen.dart`	■
Leave application	`leave_application_screen.dart`	■
My timetable (+ PDF share)	`my_timetable_screen.dart`	■
Marks / test marking	`marks_entry_screen.dart`, `test_marking_screen.dart`, `task_marking_screen.dart`	■ overlapping marks/test/task screens — confirm which are live
Daily parent calls	`daily_calls_screen.dart`	■
Staff tasks (assigned)	`staff_tasks_screen.dart`, `task_status_screen.dart`, `task_badge_widgets.dart`	■
Meeting tasks	`meeting/teacher_meeting_tasks_screen.dart`	■
Student list / details / remarks	`student_list_screen.dart`, `student_details_screen.dart`, `student_remarks_screen.dart`, `student_selection_screen.dart`	■
Notifications	`notifications_screen.dart`	■

Guardian (`guardian`)

Feature	Screen(s)	Status
Guardian login (email/pw + phone OTP)	`guardian_login_screen.dart`, `phone_otp_screen.dart`	■
Guardian dashboard	`guardian_dashboard.dart`	■
Child student details	`guardian_student_details_screen.dart`	■
Submit child leave	`guardian_leave_application_screen.dart`	■
Parental consent flow	`consent/parental_consent_flow.dart` + `widgets/consent_pending_banner.dart`	■
Birthdays	`birthdays/birthdays_screen.dart`	■

Cross-cutting / shared

- **Auth & routing:** `main.dart` `_SplashGate`, `login_screen.dart`, `role_selection_screen.dart`, `forgot_password_screen.dart`, `utils/role_guard.dart`. There are **multiple login entry points** (`login_screen`, `role_selection_screen`, `guardian_login_screen`) — `main.dart` boots into `LoginScreen`; `role_selection_screen.dart` may be legacy (■, confirm before launch).

5. Screen Inventory

~98 Dart files under `lib/screens/`. Status reflects source presence/size, not runtime testing.

Screen file	Purpose	Status
`add_student_screen.dart`	Add a new student	■
`admin_screen.dart`	Admin utilities (users, data)	■
`admin/audit_log_screen.dart`	View immutable audit log	■
`analytics_screen.dart`	Attendance trend / absence-leader charts	■
`announcements_screen.dart`	Post/read announcements	■
`assign_duties_screen.dart`	Duty roster assignment	■

Screen file	Purpose	Status
attendance_certificate_screen.dart	Generate attendance certificate PDF	■
attendance_history_screen.dart	Calendar of past attendance	■
attendance_screen.dart	Mark daily attendance	■
birthdays/birthdays_screen.dart	Student/staff birthdays	■
class_picker_screen.dart	Pick class+section utility	■
consent/parental_consent_flow.dart	Guardian consent capture	■
coordinator/absent_teachers_screen.dart	Today's absent teachers	■
coordinator/report_card_template_editor.dart	Edit report-card templates	■
coordinator_dashboard.dart	Coordinator home	■
coordinator_staff_tasks_screen.dart	Coordinator's tasks	■
copy_check_overview_screen.dart	Copy-check status across classes	■
copy_checking_screen.dart	Log copy-check per student	■
create_task_screen.dart	Create a task	■ overlaps tasks/create_staff_task_screen.dart
daily_calls_screen.dart	Track parent calls (tel: dial)	■
exam_management_screen.dart	Create/manage exams	■
fee_collection_screen.dart	Record fee payments	■
fee_overview_screen.dart	Fee overview	■
fee_structure_screen.dart	Set per-class fee structure	■
forgot_password_screen.dart	Password reset	■
free_bells_screen.dart	Unassigned bell slots	■
gallery/album_detail_screen.dart	Photos in an album, upload	■
gallery/create_album_screen.dart	Create album	■
gallery/fullscreen_photo_viewer.dart	Full-screen photo viewer	■
guardian_dashboard.dart	Guardian home	■
guardian_leave_application_screen.dart	Guardian submits child leave	■
guardian_login_screen.dart	Guardian login (email/phone)	■
guardian_student_details_screen.dart	Child details	■
home_screen.dart	Teacher home	■
homework_overview_screen.dart	All homework overview	■
homework_screen.dart	Post/manage homework	■
leave_application_screen.dart	Teacher leave request	■
leave_requests_screen.dart	Approve/reject leave	■
login_screen.dart	Primary login (Firebase Auth)	■
marks_entry_screen.dart	Enter exam marks	■
meeting/coordinator_meeting_records_screen.dart	Coordinator meeting records	■
meeting/meeting_detail_screen.dart	Single meeting detail	■

Screen file	Purpose	Status
meeting/principal_meeting_records_screen.dart	Principal meeting records	■
meeting/teacher_meeting_tasks_screen.dart	Teacher meeting tasks	■
my_timetable_screen.dart	Personal timetable + PDF	■
notifications_screen.dart	Notification feed	■
onboarding/school_onboarding_screen.dart + step1…6	6-step school setup wizard	■ see §9 multi-tenant
owner/owner_home.dart	Owner dashboard	■
owner/owner_principal_home.dart	Combined owner+principal home	■
owner/edit_school_settings_screen.dart	Edit school config/branding	■
phone_otp_screen.dart	Phone OTP verification	■
principal_dashboard.dart	Principal home	■
principal_digest_screen.dart	EOD digest + PDF	■
report_card_screen.dart	Report card + rank/grade/PDF	■
role_selection_screen.dart	Role-based login UI	■ likely legacy (boot path uses LoginScreen)
staff_task_management_screen.dart	Principal task management	■
staff_tasks_screen.dart	Staff task list	■
student_deletion_requests_screen.dart	Review delete requests	■
student_details_screen.dart	View/edit student	■
student_leave_requests_screen.dart	Student leave queue	■
student_list_screen.dart	Class roster	■
student_remarks_screen.dart	Student remarks	■
student_selection_screen.dart	Pick a student	■
substitution_history_screen.dart	Past substitutions	■
substitution_plan_screen.dart	Auto-suggest substitutions	■
task_badge_widgets.dart	Task badge UI widgets	■ (widget lib)
task_marking_screen.dart	Mark task completion	■ overlaps test/marks screens
task_status_screen.dart	Task status view	■
tasks/create_staff_task_screen.dart	Create staff task	■
tasks/staff_task_analytics_view.dart	Task analytics	■
tasks/staff_task_detail_screen.dart	Task detail	■
teacher_management_screen.dart	Add/edit/remove teachers	■
test_marking_screen.dart	Enter test marks	■ overlaps marks/task screens
timetable_settings_screen.dart	Bells + class list config	■
todo_list_screen.dart	To-do list	■
todo_reminder_banner.dart	To-do reminder banner	■ (widget)

Several SPEC-era screens (timetable_editor_screen, bell_settings_screen, class_management_screen, class_selection_screen, scan_students_screen, teacher_profile_screen, subject_teacher_home, teacher_dashboard_screen, coordinator_home, principal_home, guardian_home, guardian_portal_screen, reports_screen, attendance_class_detail_screen, student_profile_screen, test_creation_screen,

`timetable_screen`, `history_screen`, `auth_gate`) **no longer exist** in `lib/screens/` — they were removed/renamed since `SPEC.md`.

6. Firebase

Services actually used

Service	Used?	Evidence
Firebase Core	■	`firebase_core`; `Firebase.initializeApp` in `main.dart`
Firebase Auth	■	`firebase_auth`; email/pw + phone OTP + password reset in `auth_service.dart`, `login_screen.dart`, `phone_otp_screen.dart`
Cloud Firestore	■	`cloud_firestore`; all services
Firebase Storage	■	`firebase_storage`; gallery photo upload (`gallery_service.dart`)
FCM / Messaging	■	No `firebase_messaging` package. Notifications are **Firestore documents**, not push. Dead `saveFcmToken()` in `firestore_service.dart` writes to a `users` collection with no messaging behind it.
Analytics	■	Not present
Crashlytics	■	Not present
Dynamic Links	■	Not present

Firestore data model

Identity is a **root collection**; everything else is **school-scoped under** `schools/{sid}/…`.

Collection (path)	Key fields	Relationships
`allowed_users/{uid}`	`role`, `schoolId`, `classIds[]`, `studentIds[]`, `status`, `name`, `email`	uid = Firebase Auth UID. `schoolId` scopes all other reads. `classIds`/`studentIds` gate guardian/teacher access.
`schools/{sid}`	school root doc (branding, name)	parent of all scoped data
`schools/{sid}/students/{studentId}`	`className`, `section`, roll, parent fields, `guardianDetails`, `guardianEmail`	id = `{className}_{section}_{roll}`. Guardian access via `studentIds[]`.
`…/students/{id}/consents/{cid}`	consent record, `withdrawnAt/Reason/By`	child of student; immutable (no delete)
`…/students/{id}/remarks/{rid}`	remark text, author, ts	read via **collection-group** by management
`schools/{sid}/consent_withdrawals/{wfId}`	withdrawal workflow	management-actioned
`schools/{sid}/teachers/{teacherId}`	name, subject, class-teacher flags	referenced by timetable/leave/tasks
`schools/{sid}/attendance/{cls}`	doc id = class name; date keys → roll→status sub-map	guardian read gated by `classIds[]`
`schools/{sid}/announcements/{id}`	title, body, audience, pinned	all school members read
`schools/{sid}/leave_applications/{id}`	`teacherId`, `status`, review fields	teacher self + management
`schools/{sid}/fees/{feeId}`	`className`, components, total	guardian read via `classIds[]`
`schools/{sid}/fee_payments/{id}` (+ `…/payments` subcols)	`studentId`, amount, date	guardian read via `studentIds[]`; collection-group `payments` read by management
`schools/{sid}/exams/{examId}`	`className`, subjects, maxMarks	**all school members read** (see assessment)

Collection (path)	Key fields	Relationships
`schools/{sid}/exam_results/{resultId}`	`className`, roll, marks, grade	**all school members read** (see assessment)
`schools/{sid}/copy_checks/{checkId}`	`className`, statuses	**all school members read**
`schools/{sid}/staff_tasks/{taskId}`	`assignedTo`, `status`, checkpoints	assignee + management
`schools/{sid}/notifications/{notifId}`	`audience` (`all/teachers/guardians/coordinator/principal/teacher:{uid}/guardian:{class}:{roll}/class_teacher:{class}`)	audience-filtered reads
`schools/{sid}/timetable/{docId}`	class timetable	management write
`schools/{sid}/settings/{docId}`	bells, classes, school config	management write
`schools/{sid}/duties/{docId}`	duty roster	management write
`schools/{sid}/substitutions/{docId}`	active substitution plans	management write
`schools/{sid}/homework/{docId}`	homework	teachers + management write
`schools/{sid}/audit_logs/{logId}`	action/entity/before/after	append-only; management read; soft-delete only
`schools/{sid}/report_card_templates/{id}`	grade bands, GPA	management write
`schools/{sid}/albums/{albumId}`	title, `published`, coverUrl	published → public read
`schools/{sid}/photos/{photoId}`	urls, `published`	published → public read
`schools/{sid}/meetings/{meetingId}`	agenda, points, assignees	staff read; management write
`schools/{sid}/meetingTasks/{taskId}`	`assignedTo`, `status`	assignee + management

Storage: `schools/{schoolId}/gallery/{albumId}/{original|compressed|watermarked}/{photoId}.jpg`. `GalleryService` still hardcodes `_schoolId = 'school_1'`.

`firestore.rules` *(verbatim, current)*

```
rules_version = '2';

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// School App — Firestore Security Rules
//
// Architecture notes
// ■■■■■■■■■■■■■■■■■■■■■■■
// • All user identity lives in  allowed_users/{uid}  (root, not school-scoped)
//   Fields: role, schoolId, classIds[], studentIds[], status, name, email
// • Every other collection is school-scoped: schools/{sid}/...
// • Roles: teacher · subjectTeacher · guardian · coordinator · principal
//          admin   · owner
// • Attendance doc ID  = class name string  e.g. "Class 9-A"
// • Student  doc ID    = "{cls}_{section}_{roll}" (spaces → underscores)
// • Notification audience field formats — see NOTIFICATIONS block below
//
// Hard guarantees
// ■■■■■■■■■■■■■■■■■■■
// ① Default-deny catch-all at the top of every database match.
// ② Every school-scoped rule calls inSchool(sid) first.
// ③ Role escalation blocked: non-admin self-update cannot change role or schoolId.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

service cloud.firestore {
  match /databases/{database}/documents {

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // ① DEFAULT DENY — every path must carry an explicit allow.
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    match /{document=**} {
      allow read, write: if false;
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
// HELPER FUNCTIONS
//
// userDoc() result is memoised per evaluation — multiple calls in
// a single rule cost only ONE document-read quota unit.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

/// Firebase Auth token is present.
function isSignedIn() {
  return request.auth != null;
}

/// The calling user's allowed_users document.
function userDoc() {
  return get(/databases/$(database)/documents/allowed_users/$(request.auth.uid));
}

/// Caller's role string (e.g. 'teacher', 'coordinator', …).
function userRole() {
  return userDoc().data.role;
}

/// Caller's schoolId.
function userSchoolId() {
  return userDoc().data.schoolId;
}

/// True when caller's role exactly equals r.
function isRole(r) {
  return userRole() == r;
}

/// ② True when the caller's schoolId matches path variable sid.
function inSchool(sid) {
  return userSchoolId() == sid;
}

/// True when cls is listed in the caller's classIds array.
function isClassTeacher(cls) {
  return cls in userDoc().data.classIds;
}

/// True when caller's role is one of the given list literal.
function hasAnyRole(roles) {
  return userRole() in roles;
}

// ■■ Convenience role groups ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

/// coordinator | principal | admin | owner
function isManagement() {
  return hasAnyRole(['coordinator', 'principal', 'admin', 'owner']);
}

/// admin | owner  (user-lifecycle / privileged operations)
function isStrictAdmin() {
  return hasAnyRole(['admin', 'owner']);
}

/// teacher | subjectTeacher
function isAnyTeacher() {
  return hasAnyRole(['teacher', 'subjectTeacher']);
}

/// Any staff member (teachers + management).
function isAnyStaff() {
  return hasAnyRole(['teacher', 'subjectTeacher',
                     'coordinator', 'principal', 'admin', 'owner']);
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ALLOWED USERS  (/allowed_users/{uid})
//
// Root collection — identity is cross-school by design. schoolId
// embedded in each document scopes reads and writes.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /allowed_users/{uid} {

  // Own document, OR any coordinator / principal / admin in the same school.
  allow read: if isSignedIn() && (
    request.auth.uid == uid ||
    (isManagement() && userSchoolId() == resource.data.schoolId)
  );

  // ③ Only strict admin; new document must belong to caller's school.
  allow create: if isSignedIn()
```

```
    && isStrictAdmin()
    && userSchoolId() == request.resource.data.schoolId;

  allow update: if isSignedIn() && (
    // Admin: full update power within own school.
    (isStrictAdmin() && userSchoolId() == resource.data.schoolId) ||

    // ③ Non-admin self-update: role and schoolId MUST remain unchanged.
    (request.auth.uid == uid
      && request.resource.data.role     == resource.data.role
      && request.resource.data.schoolId == resource.data.schoolId)
  );

  allow delete: if isSignedIn()
    && isStrictAdmin()
    && userSchoolId() == resource.data.schoolId;
}


// ═══════════════════════════════════════════════════════════════════════
// SCHOOL ROOT DOCUMENT  (/schools/{sid})
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isStrictAdmin();
}


// ═══════════════════════════════════════════════════════════════════════
// STUDENTS  (/schools/{sid}/students/{studentId})
//
// Document ID = "{className}_{section}_{roll}" (spaces → underscores)
// Guardian access is gated on studentId appearing in their studentIds[].
// Guardian may self-serve guardianDetails for their own child.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/students/{studentId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isAnyStaff() ||
    (isRole('guardian') && studentId in userDoc().data.studentIds)
  );

  allow create: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(request.resource.data.className))
  );

  allow update: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(resource.data.className)) ||
    // Guardian: only guardianDetails / guardianEmail for their own child.
    (isRole('guardian')
      && studentId in userDoc().data.studentIds
      && request.resource.data.diff(resource.data).affectedKeys()
           .hasOnly(['guardianDetails', 'guardianEmail']))
  );

  allow delete: if isSignedIn() && inSchool(sid) && isManagement();


  // ── PARENTAL CONSENTS  (/schools/{sid}/students/{sid}/consents/{cid})
  //
  // Read:   own guardian (studentId in their studentIds[]) or any management.
  // Create: any authenticated school member (service writes on enrollment).
  // Update: guardian (own child) — only withdrawnAt/withdrawnReason/withdrawnBy
  //         fields (soft withdrawal). Management may also update.
  // Delete: nobody — immutable audit trail; use withdrawnAt instead.
  // ═══════════════════════════════════════════════════════════════════════
  match /consents/{consentId} {

    allow read: if isSignedIn() && inSchool(sid) && (
      isManagement() ||
      (isRole('guardian') && studentId in userDoc().data.studentIds)
    );

    allow create: if isSignedIn() && inSchool(sid);

    allow update: if isSignedIn() && inSchool(sid) && (
      isManagement() ||
      // Guardian may only stamp withdrawal fields
      (isRole('guardian')
        && studentId in userDoc().data.studentIds
        && request.resource.data.diff(resource.data).affectedKeys()
             .hasOnly(['withdrawnAt', 'withdrawnReason', 'withdrawnBy']))
    );
    allow delete: if false;
```

```
  }
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// CONSENT WITHDRAWALS  (/schools/{sid}/consent_withdrawals/{wfId})
//
// Workflow documents created when a guardian withdraws consent.
// Read/update: management only (admin must review and action).
// Create: any authenticated school member (guardian + service layer).
// Delete: nobody.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/consent_withdrawals/{wfId} {
  allow read, update: if isSignedIn() && inSchool(sid) && isManagement();
  allow create:       if isSignedIn() && inSchool(sid);
  allow delete:       if false;
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// TEACHERS  (/schools/{sid}/teachers/{teacherId})
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/teachers/{teacherId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ATTENDANCE  (/schools/{sid}/attendance/{cls})
//
// Document ID = class name string, e.g. "Class 9-A".
// Guardian: restricted to classes in their classIds[].
// Write: class teacher of that class, or management.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/attendance/{cls} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isAnyStaff() ||
    (isRole('guardian') && cls in userDoc().data.classIds)
  );

  allow write: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(cls))
  );
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ANNOUNCEMENTS  (/schools/{sid}/announcements/{id})
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/announcements/{announcementId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// LEAVE APPLICATIONS  (/schools/{sid}/leave_applications/{leaveId})
//
// Read:   submitting teacher (teacherId == uid) + management.
// Create: any teacher; teacherId MUST equal their own uid.
// Update: management → status/review fields only.
//         Teacher (own pending application) → everything except status.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/leave_applications/{leaveId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && resource.data.teacherId == request.auth.uid)
  );

  // No impersonation: teacherId must equal the calling user's uid.
  allow create: if isSignedIn() && inSchool(sid)
    && isAnyTeacher()
    && request.resource.data.teacherId == request.auth.uid;

  allow update: if isSignedIn() && inSchool(sid) && (

    // Management may only update status + review metadata.
    (isManagement()
      && request.resource.data.diff(resource.data).affectedKeys()
           .hasOnly(['status', 'reviewedBy', 'reviewedAt', 'remarks'])) ||

    // Submitting teacher may edit body fields of their own PENDING leave
```

```
    // but may NOT touch status or reassign teacherId.
    (isAnyTeacher()
      && resource.data.teacherId == request.auth.uid
      && resource.data.status == 'pending'
      && !request.resource.data.diff(resource.data).affectedKeys()
            .hasAny(['status', 'teacherId']))
  );

  allow delete: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// FEES  (/schools/{sid}/fees/{feeId})
//
// Class-level FeeStructure documents.
// Guardian reads limited to their children's class (via classIds[]).
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/fees/{feeId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isRole('guardian') && resource.data.className in userDoc().data.classIds)
  );

  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// FEE PAYMENTS  (/schools/{sid}/fee_payments/{paymentId})
//
// Per-student payment records.
// Guardian reads limited to their own children (via studentIds[]).
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/fee_payments/{paymentId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isRole('guardian') && resource.data.studentId in userDoc().data.studentIds)
  );

  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// EXAMS  (/schools/{sid}/exams/{examId})
//
// Readable by every school member (including guardians).
// Write: class teacher of the exam's className, or management.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/exams/{examId} {
  allow read: if isSignedIn() && inSchool(sid);

  allow create: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(request.resource.data.className))
  );

  allow update: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(resource.data.className))
  );

  allow delete: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// EXAM RESULTS  (/schools/{sid}/exam_results/{resultId})
//
// Same access pattern as exams.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/exam_results/{resultId} {
  allow read: if isSignedIn() && inSchool(sid);

  allow create: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(request.resource.data.className))
  );

  allow update: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(resource.data.className))
  );
```

```
  allow delete: if isSignedIn() && inSchool(sid) && isManagement();
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// COPY CHECKS  (/schools/{sid}/copy_checks/{checkId})
//
// Same access pattern as exams.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/copy_checks/{checkId} {
  allow read: if isSignedIn() && inSchool(sid);

  allow create: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(request.resource.data.className))
  );

  allow update: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (isAnyTeacher() && isClassTeacher(resource.data.className))
  );

  allow delete: if isSignedIn() && inSchool(sid) && isManagement();
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// STAFF TASKS  (/schools/{sid}/staff_tasks/{taskId})
//
// Read:   assignee OR management.
// Create/Delete: principal | admin | owner only.
// Update: principal/admin/owner (full) or assignee (status fields only).
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/staff_tasks/{taskId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    request.auth.uid == resource.data.assignedTo
  );

  allow create, delete: if isSignedIn() && inSchool(sid)
    && hasAnyRole(['principal', 'admin', 'owner']);

  allow update: if isSignedIn() && inSchool(sid) && (
    hasAnyRole(['principal', 'admin', 'owner']) ||

    // Assignee may only update status and completion timestamps.
    (request.auth.uid == resource.data.assignedTo
      && request.resource.data.diff(resource.data).affectedKeys()
           .hasOnly(['status', 'updatedAt', 'completedAt', 'checkpoints']))
  );
}


// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// NOTIFICATIONS  (/schools/{sid}/notifications/{notifId})
//
// Audience field formats written by NotificationService:
//   'all'                    → any authenticated school member
//   'teachers'               → teacher | subjectTeacher
//   'guardians'              → any guardian
//   'coordinator'            → coordinator role
//   'principal'              → principal role
//   'teacher:{uid}'          → a specific teacher (UID match)
//   'guardian:{class}:{roll}' → class-level check; client query filters roll
//   'class_teacher:{class}'   → the class teacher of the named class
//
// split(':')[1] is safe here because class names in this app do not
// contain colon characters. If that assumption changes, switch to
// audience.replace('class_teacher:', '') or store audience parts separately.
//
// Write: any authenticated school member for now.
// TODO: lock down to Cloud Function / Admin SDK service account.
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match /schools/{sid}/notifications/{notifId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    // Broadcast
    resource.data.audience == 'all' ||

    // Group audiences
    (resource.data.audience == 'teachers'   && isAnyTeacher())      ||
    (resource.data.audience == 'guardians'  && isRole('guardian'))   ||

    // Role-targeted
    (resource.data.audience == 'coordinator' && isRole('coordinator')) ||
```

```
    (resource.data.audience == 'principal'   && isRole('principal'))   ||

    // Direct teacher — 'teacher:{uid}'
    resource.data.audience == ('teacher:' + request.auth.uid) ||

    // Guardian — 'guardian:{class}:{roll}'
    // Rules enforce class-level access; client must filter by exact roll.
    (isRole('guardian')
      && resource.data.audience.matches('guardian:.*')
      && resource.data.audience.split(':')[1] in userDoc().data.classIds) ||

    // Class teacher — 'class_teacher:{className}'
    (isAnyTeacher()
      && resource.data.audience.matches('class_teacher:.*')
      && isClassTeacher(resource.data.audience.split(':')[1]))
  );

  allow write: if isSignedIn() && inSchool(sid);
}


// ═══════════════════════════════════════════════════════════════════════
// TIMETABLE  (/schools/{sid}/timetable/{docId})
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/timetable/{docId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// SETTINGS  (/schools/{sid}/settings/{docId})
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/settings/{docId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// DUTIES  (/schools/{sid}/duties/{docId})
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/duties/{docId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// SUBSTITUTIONS  (/schools/{sid}/substitutions/{docId})
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/substitutions/{docId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// HOMEWORK  (/schools/{sid}/homework/{docId})
//
// All school members may read; any teacher or management may write.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/homework/{docId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid)
    && (isManagement() || isAnyTeacher());
}


// ═══════════════════════════════════════════════════════════════════════
// AUDIT LOGS  (/schools/{sid}/audit_logs/{logId})
//
// APPEND-ONLY — once created, records are immutable for all users.
//   Read:   principal | admin | owner | coordinator in school.
//   Create: any authenticated school member (service layer only;
//           fire-and-forget from service singletons).
//   Update: admin/owner ONLY, and only to add a soft-delete reason
//           (deletedReason, deletedAt, deletedBy fields). All other
//           fields are locked.
//   Delete: nobody — hard deletes are permanently forbidden.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/audit_logs/{logId} {

  allow read: if isSignedIn() && inSchool(sid)
    && hasAnyRole(['principal', 'admin', 'owner', 'coordinator']);
  allow create: if isSignedIn() && inSchool(sid);
```

```
  // Soft-delete only: admin may stamp a reason but cannot alter
  // any original audit fields (action, entity, before, after, etc.)
  allow update: if isSignedIn() && inSchool(sid)
    && isStrictAdmin()
    && request.resource.data.diff(resource.data).affectedKeys()
         .hasOnly(['deletedReason', 'deletedAt', 'deletedBy']);

  allow delete: if false;
}


// ═══════════════════════════════════════════════════════════════════════
// REPORT CARD TEMPLATES  (/schools/{sid}/report_card_templates/{templateId})
//
// Read: all school members.
// Write: coordinator | principal | admin | owner.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/report_card_templates/{templateId} {
  allow read:  if isSignedIn() && inSchool(sid);
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// GALLERY — Albums & Photos
// (/schools/{sid}/albums/{albumId}, /schools/{sid}/photos/{photoId})
//
// Published items are publicly readable (share links, parent portals).
// Write: coordinator | principal | admin | owner.
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/albums/{albumId} {
  allow read: if (isSignedIn() && inSchool(sid))
              || resource.data.published == true;
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}

match /schools/{sid}/photos/{photoId} {
  allow read: if (isSignedIn() && inSchool(sid))
              || resource.data.published == true;
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// MEETINGS  (/schools/{sid}/meetings/{meetingId})
//
// Coordinator records: agenda, points, assigned teachers.
// Read:  any staff member (records screens + assigned teachers).
// Write: management (coordinator creates/edits/closes meetings).
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/meetings/{meetingId} {
  allow read:  if isSignedIn() && inSchool(sid) && isAnyStaff();
  allow write: if isSignedIn() && inSchool(sid) && isManagement();
}


// ═══════════════════════════════════════════════════════════════════════
// MEETING TASKS  (/schools/{sid}/meetingTasks/{taskId})
//
// Tasks spun out of a meeting point onto a teacher.
// Read:   assignee (assignedTo == uid) OR management.
// Create/Delete: management only.
// Update: management (full) or assignee (status field only).
// ═══════════════════════════════════════════════════════════════════════
match /schools/{sid}/meetingTasks/{taskId} {

  allow read: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    request.auth.uid == resource.data.assignedTo
  );

  allow create, delete: if isSignedIn() && inSchool(sid) && isManagement();

  allow update: if isSignedIn() && inSchool(sid) && (
    isManagement() ||
    (request.auth.uid == resource.data.assignedTo
      && request.resource.data.diff(resource.data).affectedKeys()
           .hasOnly(['status']))
  );
}


// ═══════════════════════════════════════════════════════════════════════
// COLLECTION-GROUP READS — remarks & payments
//
// The Principal EOD Digest fans out unscoped collectionGroup() queries
```

```
      // over every `remarks` (schools/{sid}/students/{id}/remarks) and
      // `payments` (fee_payments/{cls}/students/{roll}/payments) subcollection.
      // A collection-group query needs a rule that matches the leaf collection
      // at ANY depth via the recursive {path=**} wildcard.
      //
      // Restricted to management. NOTE: a collection-group query spans schools,
      // so these grants are NOT school-isolated. Acceptable while the app is
      // effectively single-school; revisit (stamp schoolId on each doc and
      // filter) before true multi-tenant rollout.
      // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
      match /{path=**}/remarks/{remarkId} {
        allow read: if isSignedIn() && isManagement();
      }

      match /{path=**}/payments/{paymentId} {
        allow read: if isSignedIn() && isManagement();
      }

    } // /databases/{database}/documents
  } // service cloud.firestore
```

Honest assessment: can a guardian read another student's data?

For the directly student-scoped collections, no — the rules are sound. A guardian can only read `students/{id}`, `fee_payments`, and a student's `consents` when the `studentId`/`className` is in their own `studentIds[]`/`classIds[]`. Identity (`role`, `schoolId`) cannot be self-escalated (guarantee ③). `remarks`/`payments` collection-group reads are management-only.

However, there are real cross-student leaks a reviewer must address:

1 **Exam results, exams, and copy-checks are readable by *every* school member, including guardians (`allow read: if isSignedIn() && inSchool(sid)` — no `studentIds`/`classIds` gate). A guardian who queries `schools/{sid}/exam_results` directly can read** every student's marks and grades in the entire school**, not just their child's. This is the most significant privacy gap.

1 **Attendance is class-level, not student-level.** A guardian whose `classIds[]` contains the child's class can read the *whole class* attendance document — which contains the roll→status map for **every student in that class**, not only their child. The per-roll filtering happens client-side.

1 **Guardian notifications are gated only at the class level.** For `guardian:{class}:{roll}` audiences the rule checks only that `{class}` is in the guardian's `classIds[]`; the rule comment explicitly states *"client must filter by exact roll."* A guardian can therefore read notifications addressed to **other students' parents in the same class** by bypassing the client filter.

1 `notifications` **and several workflow collections allow** `write` **by any authenticated school member** (the rule even carries a `TODO: lock down to Cloud Function`). A guardian could forge notifications. Same broad-create pattern applies to `consents`, `consent_withdrawals`, and `audit_logs` (`create` by any school member) — acceptable for service-layer fire-and-forget, but not write-restricted.

1 **Collection-group `remarks`/`payments` rules are not school-isolated** (acknowledged in the rules comment). They are management-only, so not a guardian leak, but they *do* let one school's management read another school's remarks/payments — a true multi-tenant isolation bug to fix before any second school is onboarded.

Bottom line: child-record reads are correctly scoped, but **exam results / attendance maps / class-level notifications expose other students' data to guardians**, and the multi-tenant isolation is incomplete. These need rule changes (or schema changes stamping `studentId` on exam/result docs and per-roll attendance) before a privacy-sensitive Play Store launch.

7. Android Config

Setting	Value	Source
`compileSdk`	**35**	`android/app/build.gradle`
`targetSdk`	**35**	`android/app/build.gradle`
`minSdkVersion`	**21**	`android/app/build.gradle`
`applicationId`	`com.example.school_app`	`android/app/build.gradle`
`namespace`	`com.example.school_app`	`android/app/build.gradle`

Setting	Value	Source
Java / Kotlin target	1.8	`compileOptions` / `kotlinOptions`
AGP / Kotlin / google-services	8.3.2 / 1.9.23 / 4.4.1	`android/settings.gradle`

Signing. ■■ No release signing config exists. The `release` build type explicitly sets `signingConfig signingConfigs.debug` with the comment *"Signing with the debug keys for now."* There is **no** `android/key.properties` and no keystore wiring. A release AAB built today is **debug-signed and cannot be uploaded to Play**.

Code shrinking. ■ Enabled for release: `minifyEnabled true`, `shrinkResources true`, with `proguard-android-optimize.txt` + a project `proguard-rules.pro` (keeps `io.flutter.**`, Firebase, and GMS classes). ProGuard rules look adequate for the Flutter+Firebase stack.

`applicationId` **is the Flutter default** (`com.example.…`, flagged by the auto-generated `// TODO: Specify your own unique Application ID`). `google-services.json` is registered under the same `com.example.school_app` package, so the Firebase Android app must be re-created (or have a new app added) once the real applicationId is chosen.

Permissions (`android/app/src/main/AndroidManifest.xml`)

Permission	Justifying feature	Verdict
`INTERNET`	Firebase / all network I/O	■ required
`ACCESS_NETWORK_STATE`	`connectivity_plus` offline detection	■ required
`VIBRATE`	Haptics / notifications feedback	■ verify something actually vibrates; minor
`READ_EXTERNAL_STORAGE` (`maxSdkVersion=32`)	`image_picker` gallery selection on ≤ API 32	■ scoped correctly
`WRITE_EXTERNAL_STORAGE` (`maxSdkVersion=29`)	`image_gallery_saver` saving photos on ≤ API 29	■ scoped correctly
`CAMERA`	`image_picker` camera capture + `google_mlkit_text_recognition` OCR	■ required

`uses-feature camera` / `camera.autofocus` are both `required="false"` (correct — camera is optional). `<queries>` declares `PROCESS_TEXT`, `tel:` dialing (Daily Calls), `https` viewing, and WhatsApp packages (`com.whatsapp`, `com.whatsapp.w4b`) for share/contact flows. No obviously unused/dangerous permissions. No media/notification (`POST_NOTIFICATIONS`) permission — consistent with the absence of FCM.

8. Theme

Confirmed in `lib/theme.dart` (`AppTheme`, the single colour source):

- **Material version: Material 2** — `ThemeData(useMaterial3: false)`.
- `primarySwatch: Colors.purple`, status bar transparent with light icons (edge-to-edge set in `main.dart`).
- Brand palette:
- `primary` `#6A1B9A` (Deep Violet)
- `primaryDark` `#4A148C`, `primaryMid` `#8E24AA`, `primaryLight` `#CE93D8`
- `accent` `#D81B60` (Magenta — badges/CTAs)
- `background` `#F8F0FF` (light lavender), `surface` white
- Semantic: `success` `#2E7D32`, `warning` `#F57F17`, `danger` `#C62828`
- Themed components: AppBar (violet, 0 elevation), elevated/outlined/text buttons (10px radius), FAB, inputs (10px radius), progress, checkboxes, dividers `#E0E0E0`.
- Global text scaling clamped to `0.8–1.2` in `main.dart`.

This matches the `CLAUDE.md` theme contract. (Material 3 is **not** used; a migration would be a deliberate effort, not a quick flag flip.)

9. Known Issues & Tech Debt

Source scans run: `// TODO/FIXME/HACK/XXX` in `lib/` → **none**. `print(`/`debugPrint(` in `lib/` → **21 occurrences**. Hardcoded `school_1` → **11 occurrences**. Placeholder/dummy/credential strings → none found (the few "placeholder" hits are PDF co-curricular section labels, not data).

1. **Debug/`print` logging left in production code (21 hits).** Files include `daily_calls_screen.dart`, `attendance_screen.dart`, `copy_check_overview_screen.dart`, `admin_screen.dart`, `marks_entry_screen.dart`, `principal_dashboard.dart`, `timetable_service.dart`, `audit_log_service.dart`, and `base_firestore_service.dart` (`handleError` does `print('Firestore error: …')`). Plus the migration scripts. Should be removed or routed through a logger for release.

1. **Hardcoded `'school_1'` fallback throughout the "multi-tenant" layer (11 hits).** `AuthService.currentSchoolId` → `'school_1'`; `BaseFirestoreService` fallback; `school_settings_service.dart`, `meeting_service.dart`, `gallery_service.dart` all hardcode `schoolId = 'school_1'`; `owner_home.dart` / `owner_principal_home.dart` read/write `schools/school_1/...` directly. **Multi-tenancy is scaffolded but not real** — the app is effectively single-school despite the school-scoped schema and onboarding wizard.

1. **Dead / legacy `firestore_service.dart`.** Uses a *different* attendance schema (`schools/{sid}/classes/{classId}/attendance`) than the live `StudentService`/rules path (`schools/{sid}/attendance/{cls}`), writes an unused `saveFcmToken()` to a `users` collection, and its header comment claims *"Firestore offline persistence is enabled in main.dart"* — but **no** `Settings(persistenceEnabled:…)` **exists in** `main.dart`. The `FirestoreService` class itself appears unused by screens (matches were `BaseFirestoreService`). Stale code to delete or reconcile.

1. `google_sign_in` **dependency unused** — see §2. `AuthService.signInWithGoogle()` is a `null` stub.

1. **Duplicate/overlapping screens.** `create_task_screen` vs `tasks/create_staff_task_screen`; `marks_entry_screen` vs `test_marking_screen` vs `task_marking_screen`; `login_screen` vs `role_selection_screen`. Decide canonical screens and remove the rest to avoid shipping dead UI.

1. **Half-built repository layer** (`lib/repositories/student_repository*.dart`) coexisting with the singleton-service pattern — incomplete abstraction.

1. `firestore.rules` **carries its own** `TODO`: notifications `write` should be locked down to a Cloud Function / Admin SDK; currently any school member can write notifications.

1. **One-off migration scripts shipped in** `lib/` (`scripts/migrate_users.dart`, `scripts/migrate_to_multitenant.dart`). They `print` heavily and should not be bundled in the release binary.

1. `SPEC.md` **/** `CLAUDE.md` **are stale** (claim no Firebase Auth, list merge-conflicted files and screens that no longer exist). The merge conflicts in `home_screen.dart` / `coordinator_dashboard.dart` flagged in `SPEC.md` appear **resolved** (files compile-shaped, no `<<<<<<<` markers found), but the docs themselves need updating so they don't mislead a reviewer.

> Verification limits: feature **status marks in §4–§5 are based on source presence and size, not runtime testing.** I did not launch the app or run `flutter analyze`/tests during this read-only audit.

10. Release-Readiness Gaps (for Play Store)

Blockers (must fix before upload):

1. **Set a real** `applicationId` (replace `com.example.school_app`) and re-register the Firebase Android app / `google-services.json` for that ID.

2. **Create a release signing config + upload keystore** (`key.properties` + `signingConfigs.release`). The release build is currently debug-signed and will be rejected by Play.
3. **Close the Firestore privacy leaks** (§6): gate `exam_results`/`exams`/`copy_checks` reads by `studentIds`/`classIds` for guardians (or restructure), make attendance reads child-scoped (server-side), and enforce per-roll notification access. These are data-protection issues, not cosmetics.
4. **Fix multi-tenant isolation** before onboarding a second school: replace the `'school_1'` hardcodes with the live `schoolId`, and make the collection-group `remarks`/`payments` rules school-scoped.

Strongly recommended:

1. Remove `print`/`debugPrint` from production paths; exclude `lib/scripts/**` migration tools from the release build.
2. Delete dead code/deps: `firestore_service.dart`, `google_sign_in`, duplicate screens, the half-built repository layer.
3. **App identity assets:** the launcher icon is still the **default Flutter chevron** (per the graph report's icon inventory) — ship a real branded icon set, splash, and adaptive icon.
4. **Store listing prerequisites:** privacy policy URL (the app handles children's PII, attendance, fees, photos — Play's *Families*/*Data safety* requirements apply), Data Safety form, content rating, screenshots, feature graphic.
5. Lock down `notifications` write (Cloud Function) per the in-rules TODO.
6. **No crash/observability tooling** — consider adding Crashlytics (and a deliberate decision on Analytics) before a public launch.
7. Run `flutter analyze` and the test suite (a `test/` dir exists) clean; do a real-device smoke test of each role's golden path (not done in this audit).
8. Decide on the OCR feature: `google_mlkit_text_recognition` pulls a large native dependency and the `CAMERA` permission — confirm it's actually shipped/used or drop it to slim the APK and reduce permission scope.
9. Update `SPEC.md` / `CLAUDE.md` to match reality (Firebase Auth, current screen list) so future contributors aren't misled.

Lower priority: verify `VIBRATE` is needed; review `image_gallery_saver` maintenance status; consider a Firebase dependency upgrade pass.